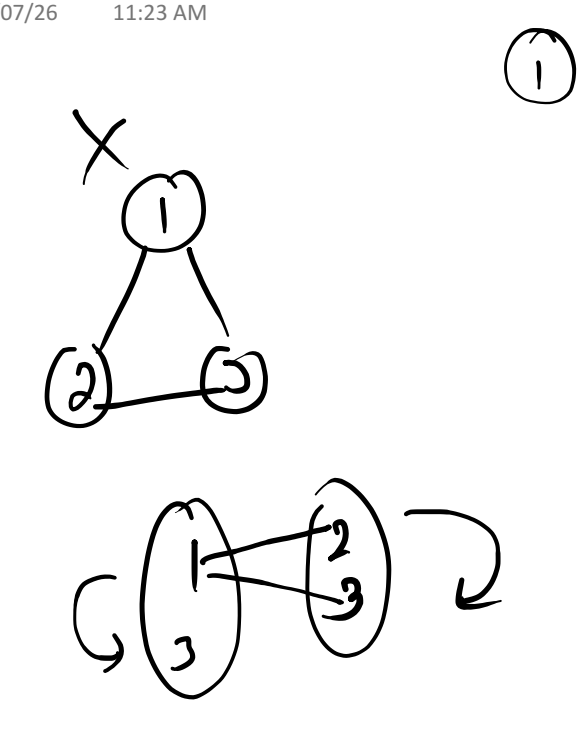
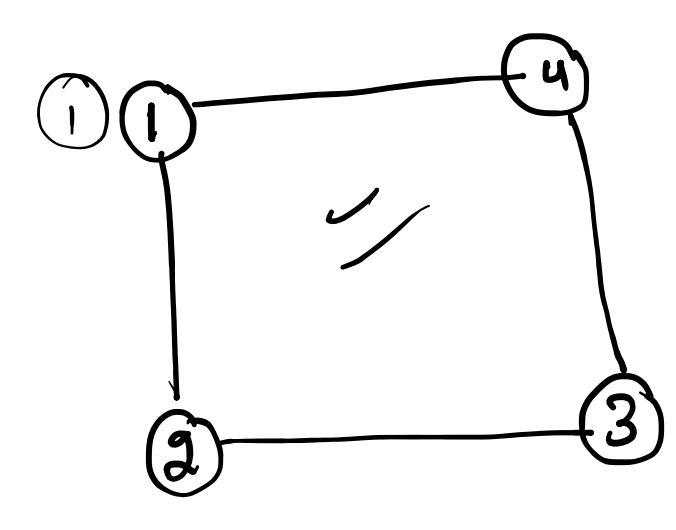
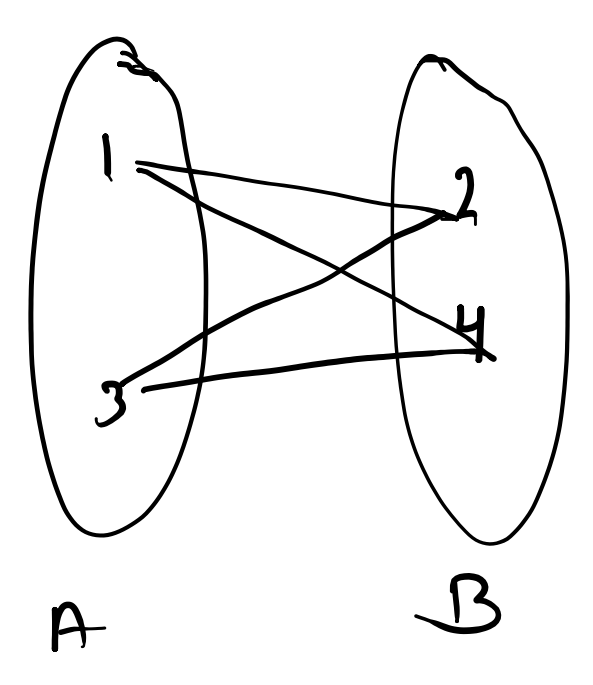
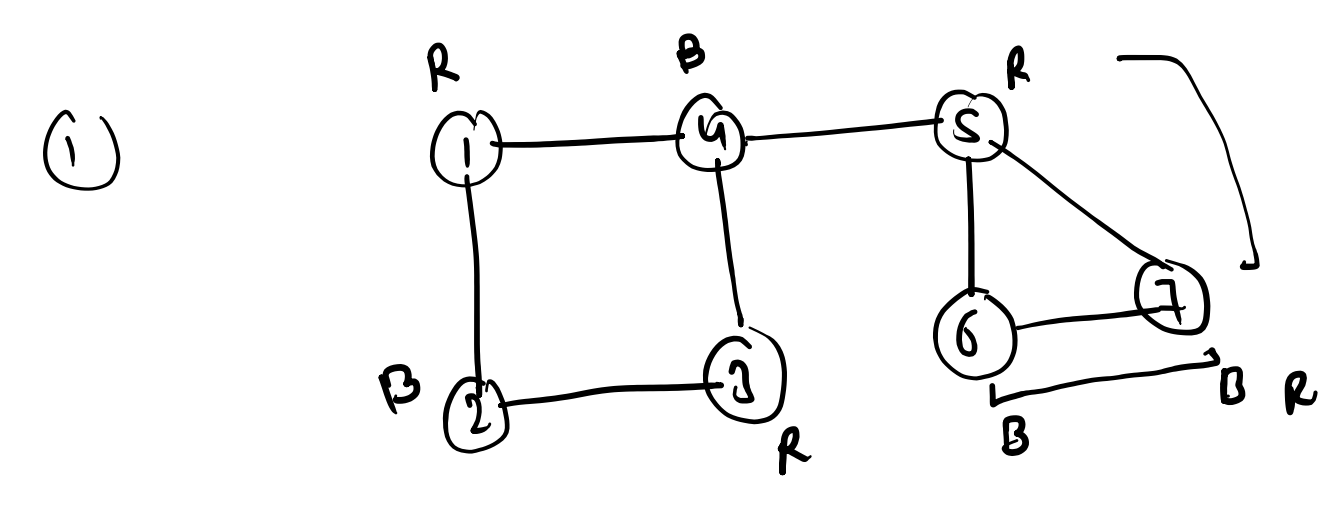
Lec-27

28/07/26 11:23 AM





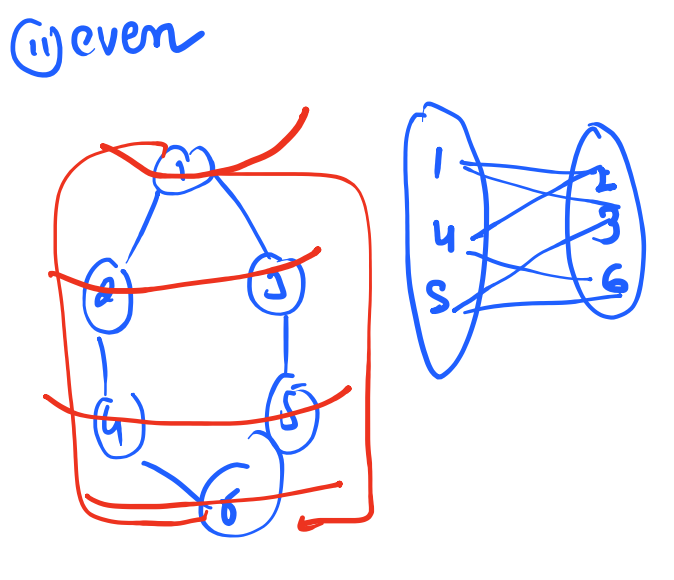



①
R
B
R
1
4
5
7
6
2
3
B
R
B
B R

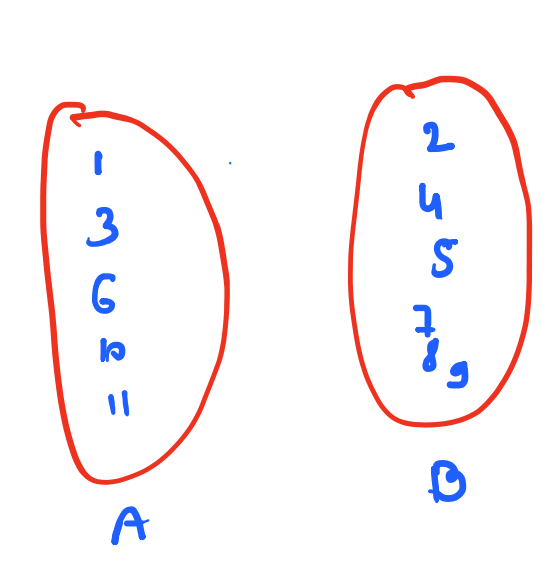
M.C.O

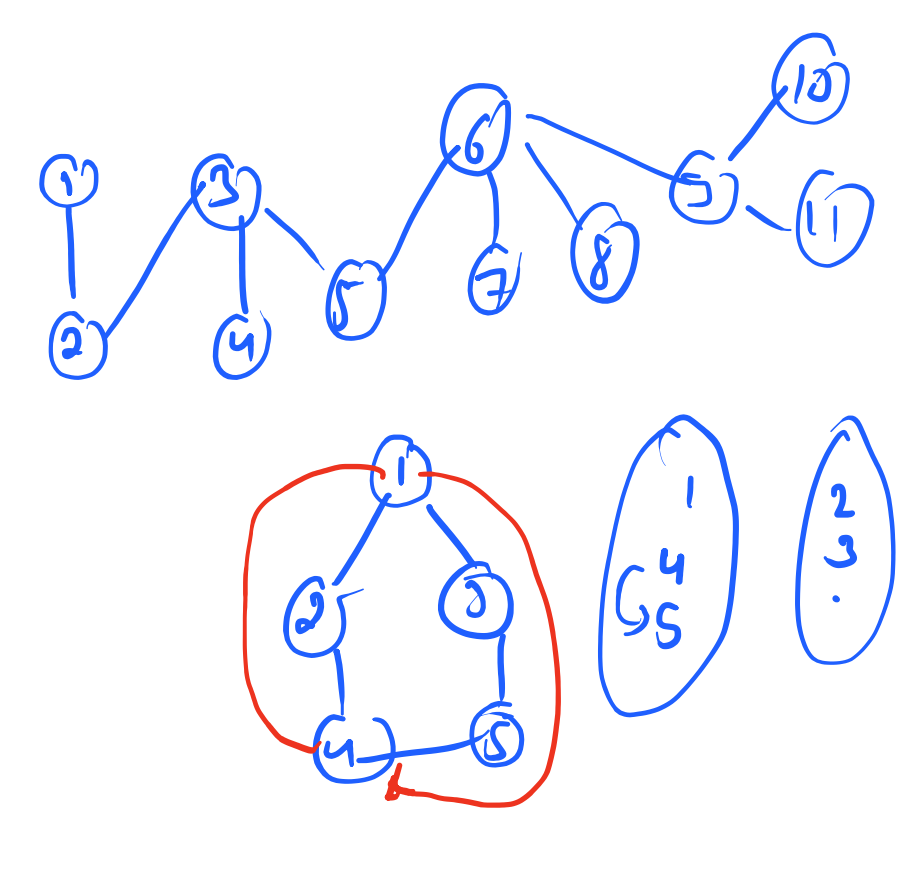
Tree -> BiP

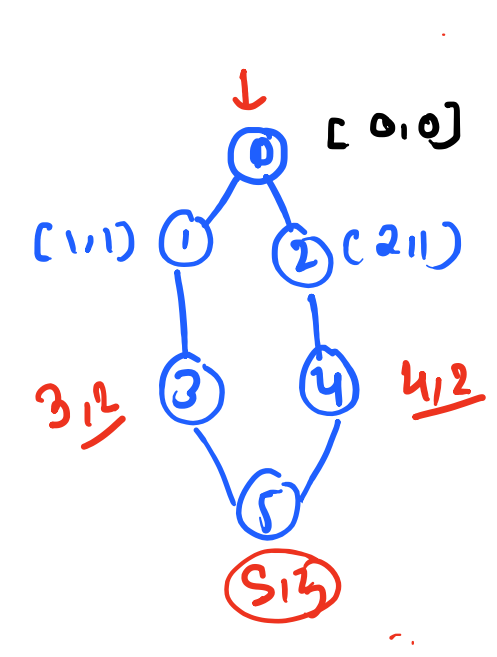
① odd
⑪ even


1
2
3
4
5
6
1
4
5
2
3
6

① cyclic   acyclic


1
3
6
10
11
2
4
5
7
8
9
A
B


1
2
3
4
5
6
7
8
9
10
11
1
2
3
4
5
1
4
5
2
3

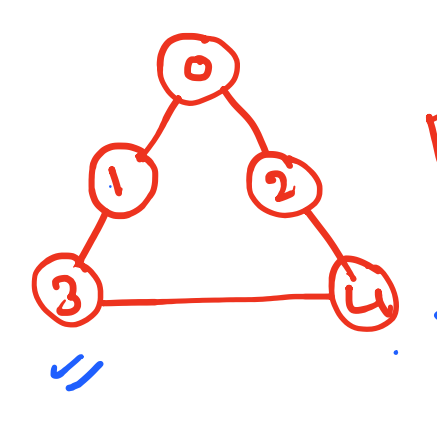

[0,0]
0
(1,1)
1
(2,1)
2
3,2
3
4,2
4
5
5,3

vtx	dis
0	0
1	1
2	1
3	2
4	2
5	2

1. remove
2. Ignore ×
3. mark visit
4. Self work
5. Add unvisited nbrs

S	
0	0
1	1
2	1
3	2
4	2

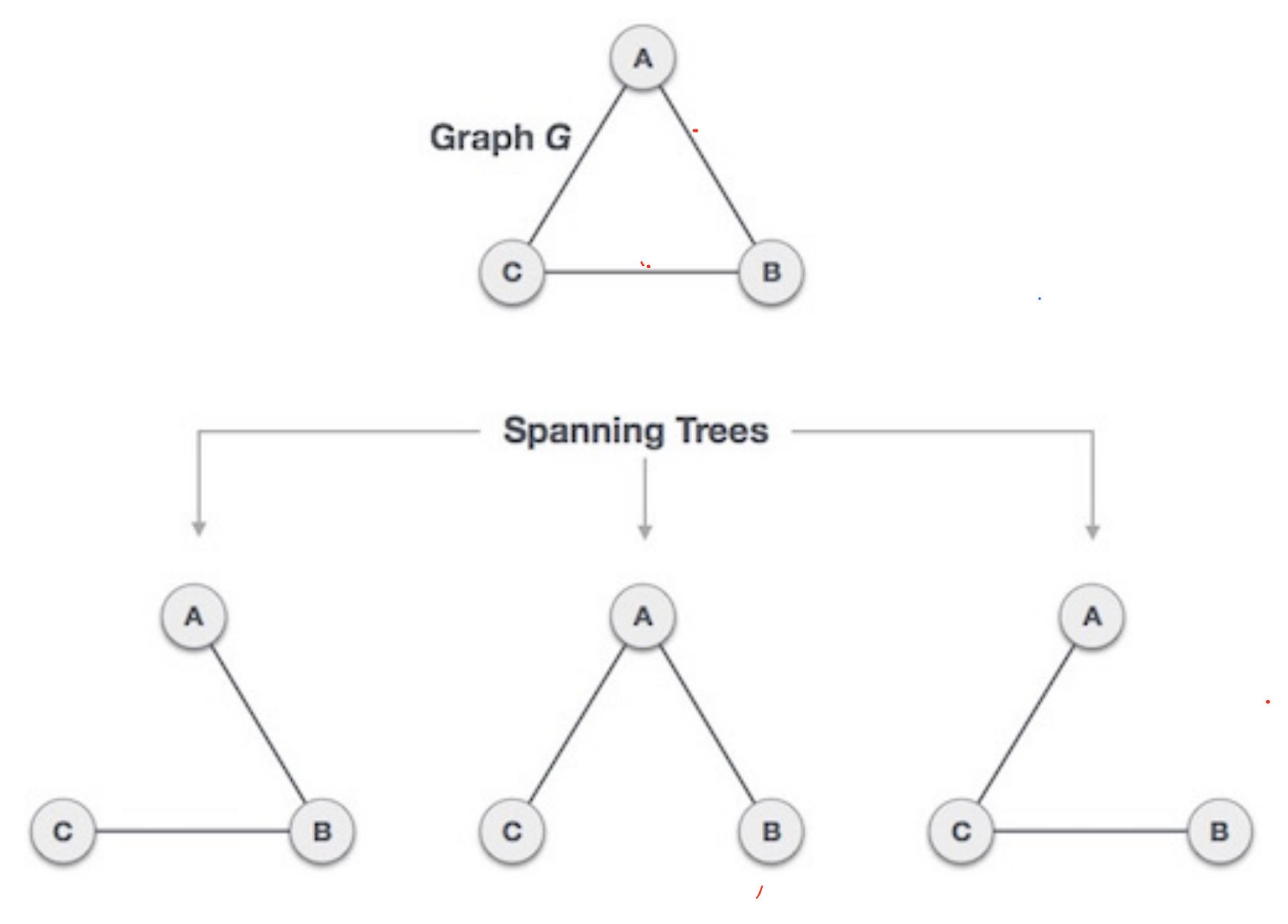
~~0,0~~	~~(1,1)~~	~~(2,1)~~	~~3,2~~	~~4,2~~	~~5,3~~	~~5,3~~



~~0,0~~	~~1,1~~	~~2,1~~	~~3,2~~	~~4,2~~	~~4,3~~

Spanning Tree

A spanning tree is a subset of Graph G, which has all the vertices covered with minimum possible number of edges. Hence, a spanning tree does not have cycles and it cannot be disconnected..


Graph G
A
C
B
Spanning Trees
A
C
B
A
C
B
A
C
B

MST

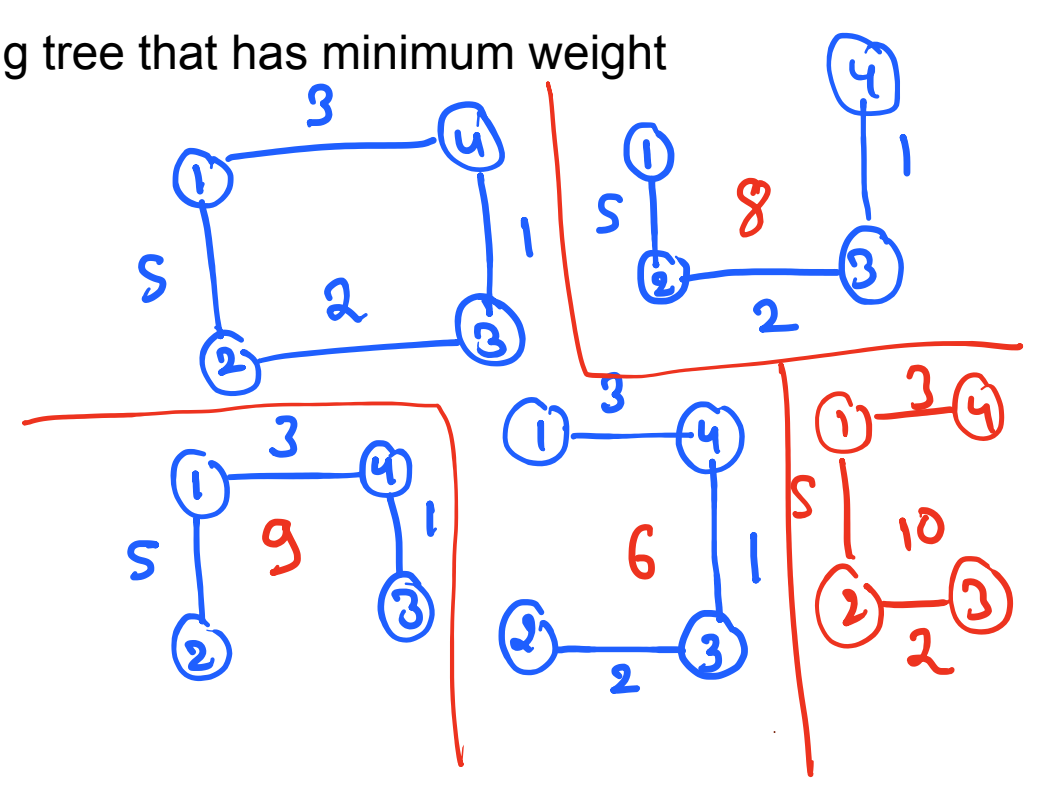
Properties of Spanning Tree

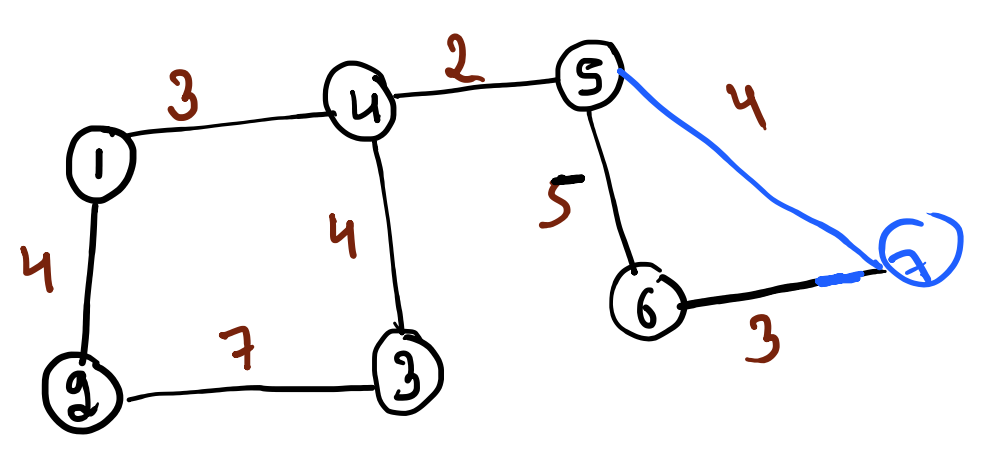
- A connected graph G can have more than one spanning tree.
- Spanning Tree has n-1 edges, where n is the number of Node (Vertices)
- All possible spanning trees of graph G, have the same number of edges and vertices.
- The spanning tree does not have any cycle (loops).
- Removing one edge from the spanning tree will make the graph disconnected, i.e. the spanning tree is minimally connected.
- Adding one edge to the spanning tree will create a circuit or loop, i.e. the spanning tree is maximally acyclic.

Minimum Spanning Tree (MST)
In a weighted graph, a minimum spanning tree is a spanning tree that has minimum weight than all other spanning trees of the same graph.

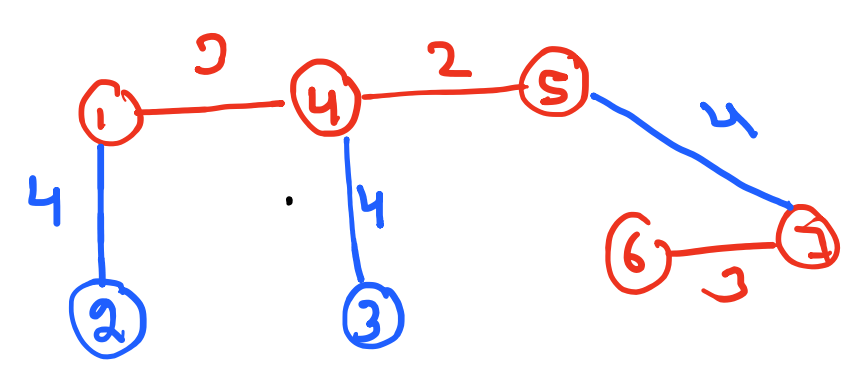
- Kruskal's Algorithm ✓

~~Prim's Algorithm~~
Both are greedy algorithms.


3
1
4
5
2
1
2
3
1
5
8
4
1
1
2
3
2
3
1
4
5
9
1
2
3
4
3
1
4
6
1
2
3
2
5
1
4
3
10
2
3
2

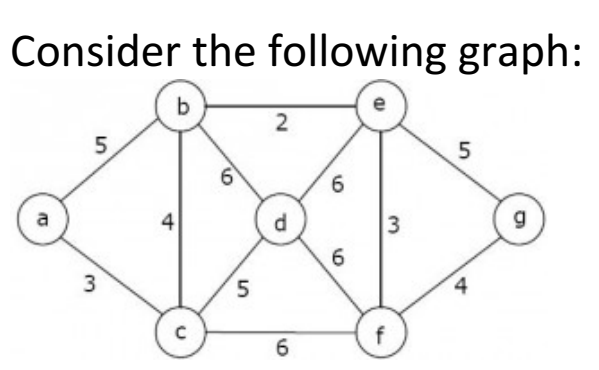

3
2
1
4
5
4
4
4
5
7
6
2
7
3
3

2+3+3+4+4+4


3
2
1
4
5
4
4
4
6
7
3
2
3

④ -> D.S.U

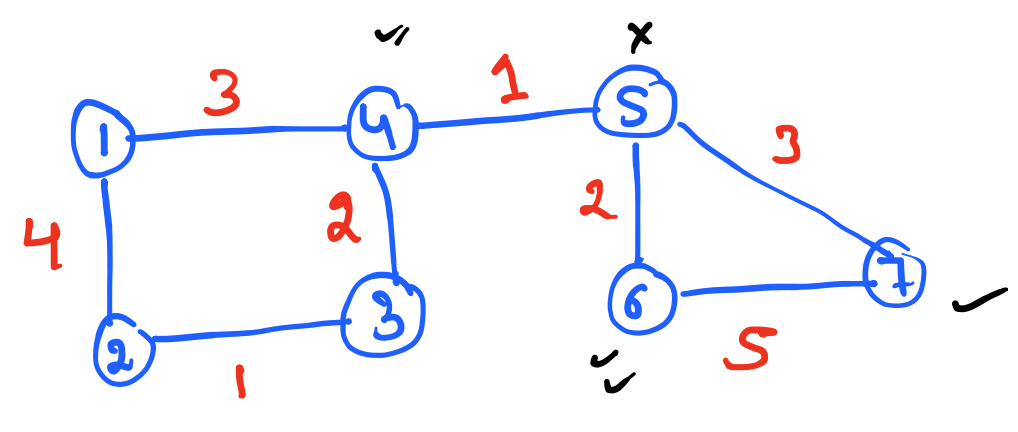
Consider the following graph:


b
e
a
d
g
c
f
5
2
6
6
5
4
3
5
6
3
4
6

Which one of the following is NOT the sequence of edges added to the minimum spanning tree using Kruskal's algorithm?
(A) (b,e)(e,f)(a,c)(b,c)(f,g)(c,d)
(B) (b,e)(e,f)(a,c)(f,g)(b,c)(c,d)
(C) (b,e)(a,c)(e,f)(b,c)(f,g)(c,d)
(D) (b,e)(e,f)(b,c)(a,c)(f,g)(c,d)

map.get(5).keySet() -> {4, 6, 7}
cost = map.get(5).get(4)
(6, 5, cost)

1. remove
2. Ignore
3. mark visited
4. Self work
5. Add nbrs

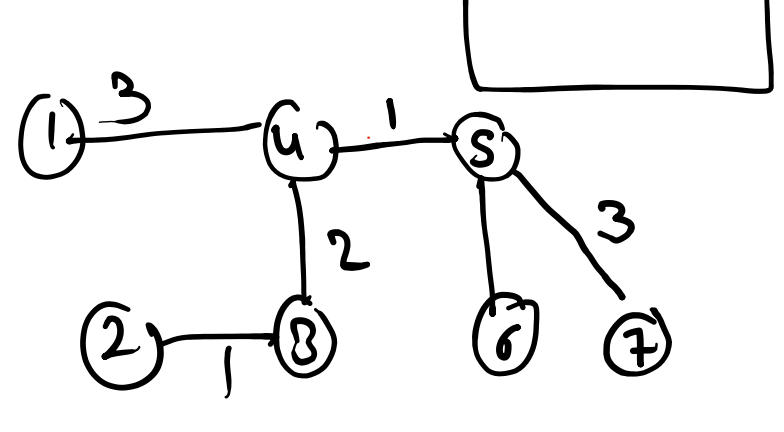

1
3
4
1
5
3
4
2
2
7
2
1
3
6
5

Prims   (V+E) log(V)

vtx	acq vtx	cost
~~7~~	~~7~~	~~0~~
~~6~~	~~7~~	~~5~~
~~5~~	~~7~~	~~3~~
~~4~~	~~5~~	~~1~~
~~6~~	~~5~~	~~2~~
~~1~~	~~4~~	~~7~~
~~3~~	~~4~~	~~2~~
~~2~~	~~3~~	~~1~~
~~1~~	~~2~~	~~4~~

7 7 @ 0
5 7 @ 3
4 5 @ 1
6 5 @ 2
3 4 @ 2
2 3 @ 1
1 4 @ 3

5 7 4 6
3 2 1


1
3
4
1
5
3
2
2
1
3
6
7

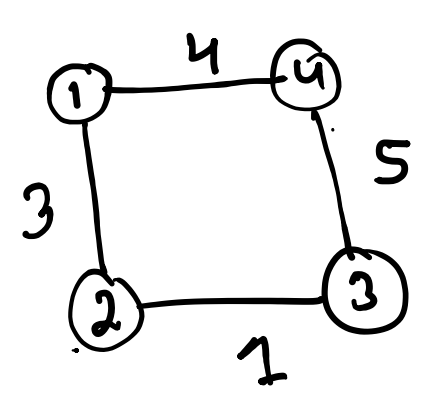
```
// 1. remove
        // 2. Ignore if Already visited
        // 3. Marked visited
        // 4. self work
        // 5. Add univisted Add nbrs
```

```
public void Prims() {
        HashSet<Integer> visited = new HashSet<>();
        PriorityQueue<PrimsPair> pq = new PriorityQueue<>((a, b) -> a.cost - b.cost);
        pq.add(new PrimsPair(1, 1, 0));
        while (!pq.isEmpty()) {
            // 1. remove
            PrimsPair rp = pq.poll();
            // 2. Ignore if Already visited
            if (visited.contains(rp.vtx)) {
                continue;
            }
            // 3. Marked visited
            visited.add(rp.vtx);
            // 4. self work
            System.out.println(rp);
            // 5. Add univisted Add nbrs
            for (int nbrs : map.get(rp.vtx).keySet()) {
                if (!visited.contains(nbrs)) {
                    int cost = map.get(rp.vtx).get(nbrs);
                    pq.add(new PrimsPair(nbrs, rp.vtx, cost));
                }
            }
        }
    }
```

{2,1,4}

1 2 3
4

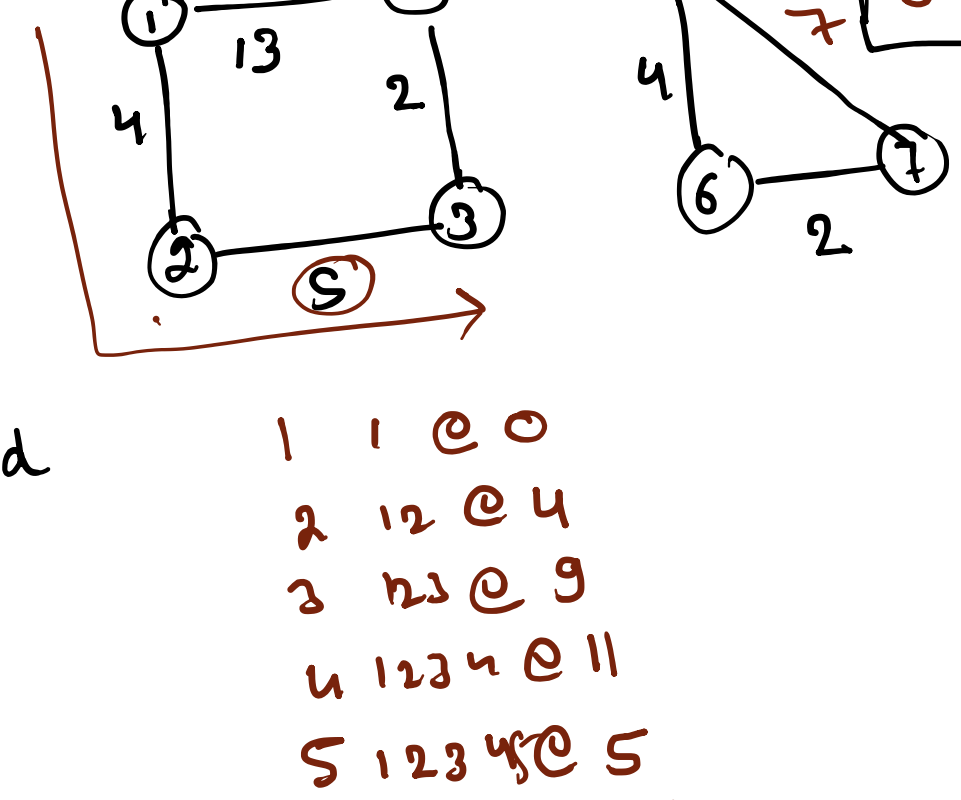
1 1 @ 0
2 2 @ 3
3 2 @ 1
4 1 @ 4



~~1~~	~~1~~	~~0~~
~~2~~	~~1~~	~~3~~
~~4~~	~~1~~	~~4~~
~~3~~	~~2~~	~~1~~
4	3	5 X

1->4 ✓
1->2 ✓
1->3 ✓
1->5 ✓
1->6 ✓
1->7 ✓

SSSP
|-> ① Dijkstra ✓
⑪ Bellman Ford


x
1
13
4
1
4
2
5
4
7
6
2
2
3
5

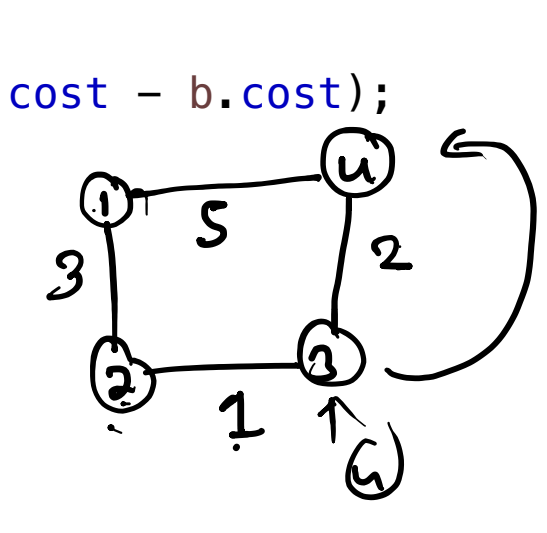
1 2 3 4
6 7

1 1 @ 0
2 12 @ 4
3 123 @ 9
4 1234 @ 11
5 12345 @ 5
6 123456 @ 16
7 1234567 @ 18

vtx	acq Path	cost
~~1~~	~~"1"~~	~~0~~
~~2~~	~~12~~	~~4~~
~~4~~	~~14~~	~~13~~
~~3~~	~~123~~	~~9~~
~~4~~	~~1234~~	~~11~~
~~5~~	~~12345~~	~~12~~
~~6~~	~~123456~~	~~16~~
~~7~~	~~12457~~	~~19~~ X
~~7~~	~~124567~~	~~18~~

```
public void Dijkstra_Algo(int src) {
        HashSet<Integer> visited = new HashSet<>();
        PriorityQueue<DijkstraPair> pq = new PriorityQueue<>((a, b) -> a.cost - b.cost);
        pq.add(new DijkstraPair(src, "" + src, 0));
        while (!pq.isEmpty()) {
            // 1. remove
            DijkstraPair rp = pq.poll();
            // 2. Ignore if Already visited
            if (visited.contains(rp.vtx)) {
                continue;
            }
            // 3. Marked visited
            visited.add(rp.vtx);
            // 4. self work
            System.out.println(rp);
            // 5. Add univisted Add nbrs
            for (int nbrs : map.get(rp.vtx).keySet()) {
                if (!visited.contains(nbrs)) {
                    int cost = map.get(rp.vtx).get(nbrs);
                    pq.add(new DijkstraPair(nbrs, rp.acqpath + nbrs, rp.cost + cost));
                }
            }
        }
    }
```

1 2 3
4



1 1 @ 0
2 12 @ 3
3 123 @ 4
4 14 @ 5

~~1~~	~~1~~	~~0~~
~~2~~	~~12~~	~~3~~
~~4~~	~~14~~	~~5~~
~~3~~	~~123~~	~~4~~
4	1234	6 X